A Multi-Agent Healthcare Triage Team

Designing & Building Agentic AI Systems — Capstone (Assignment 3)

Othmane Zizi (ID 261255341)
MMA, Desautels Faculty of Management, McGill University

June 20, 2026

Abstract

I designed and built a small but credible multi-agent system (MAS) for after-hours tele-triage on the OpenAI Agents SDK. A team of four worker agents — *intake, risk, specialist, scheduler* — is coordinated by a **supervisor** over a shared **blackboard**, with an **independent safety veto** and a **human-clinician approval gate**, to turn a synthetic patient presentation into a triage disposition (ESI acuity 1–5 + disposition band + action). The decisive design move is the *three-way split*: the model narrates, deterministic **code** owns every safety decision (red-flag screen, ESI acuity, no-down-coding, approval threshold, routing), and a **human** authorizes every high-acuity disposition. I demonstrate a named emergent failure — under-triage of an atypical heart attack — and its fix, by toggling a single governance config. Everything is synthetic; this is a decision-support prototype, not a medical device. Citations `[MAS/SDK/DRL/Intro/Harness]` refer to the course lecture decks (slide ranges).

1 System brief and why a single agent is not enough

The system answers one high-stakes question: *given an after-hours presentation, how acute is it, where should the patient go, and must a human sign off?* Stakeholders are the **patient** (safety), the **intake nurse** (completeness/throughput), the **on-call clinician** (the approval authority and a scarce resource), and **clinic operations** (capacity). The objective is to minimize **patient harm** — above all *under-triage*, missing a real emergency — subject to capacity and clinician workload. The cost of being wrong (a missed MI, stroke, or anaphylaxis) is the dominant design force `[MAS 64--73]`.

A single agent is genuinely insufficient for three reasons `[SDK 26, 61]`, `[MAS 9]`: (i) safety needs an **independent check** — the risk screener must be structurally separate from the specialist so one model's reasoning error cannot silently suppress a red flag (defense in depth; a single agent grading its own safety is the reward-hacking failure mode `[DRL 147]`); (ii) the roles have **distinct permission boundaries** (raw narrative vs. an external guideline service vs. write access to the clinic calendar); and (iii) I want a **single accountable owner** of the disposition plus an **auditable shared record**. I still honor "start with one, earn each split": each role exists only because its *contract* (tools, data, approval policy, failure cost) differs.

1.1 Agent roster, tools, and permissions

Agent	Responsibility	Tool (boundary)	Output / authority
Supervisor (code)	Routing; owns the disposition; veto + approval gate	deterministic engine; workers-as-tools	`Disposition`; gates commits
Intake	Normalize untrusted narrative; flag missing info	none (+ scope guardrail)	`IntakeSummary`; no safety authority
Risk screener	Run + relay the deterministic red-flag screen	`run_red_flag_screen`	`RiskAssessment`; the veto
Specialist	Differential + suggested ESI (advisory)	`lookup_clinical_guideline` (MCP)	`SpecialistOpinion`; advisory
Scheduler	Book a slot in the *locked* band	`book_clinic_slot` (A2A)	`SchedulingProposal`; cannot down-code
Clinician	Approve/reject high-acuity dispositions	approval UI	`ApprovalDecision`; the authority

2 Architecture and communication contract

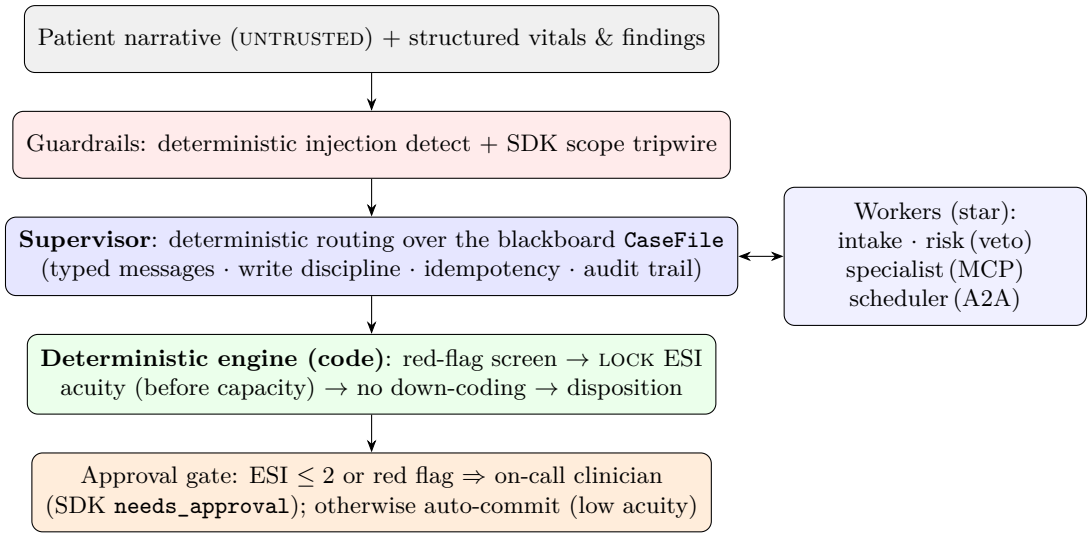

Figure 1: The supervisor + blackboard hybrid, with an independent risk veto and a human-approval gate. The model narrates within each role; deterministic code owns every safety decision; a human authorizes every high-acuity disposition.

Topology: a star, supervisor-mediated [MAS 18]. Workers never talk peer-to-peer (avoiding mesh failure modes); the supervisor routes, and every exchange is a typed `Message` posted to the blackboard `CaseFile`. The envelope carries headers (`trace_id, case_id, from/to_agent, msg_type, seq, deadline, idempotency_key, confidence, evidence`) and a **discriminated-union payload** validated against `msg_type` — a malformed message cannot be constructed, so the interaction-level "schema validity" metric is true by construction [MAS 50--61]. The blackboard enforces **write discipline** (only a section's owner or the supervisor may write it) and **idempotency** (a repeated key is dropped — exactly-once delivery, which neutralizes front-running). Escalation is a hard **stop rule**: any red flag latches a state that forces escalation regardless of consensus [Harness 126], and the bounded clarification loop terminates by construction (no message

storm).

3 Coordination mechanism — and the reasoning

I chose a **supervisor + blackboard hybrid, with an independent risk veto and a human approval gate**. Because the cost of being wrong is patient harm, I optimize for a single accountable owner, an auditable record, and an independent safety veto — not throughput `[MAS 64--73]`. Rejected alternatives: a **Contract-Net/market auction** injects a throughput incentive (auctioning patients by cost/capacity) that pushes toward under-triage — the wrong objective; **pure consensus** is slow and can converge on under-triage (groupthink), so I use it only as a tie-break that breaks toward *escalate*; **blackboard alone** has no clear owner; **supervisor alone** has no shared audit. A further, deliberate choice: the **routing itself is deterministic code**, not an LLM supervisor's discretion — control flow that can hurt a patient should not be a token prediction `[SDK 137]`. The model reasons *within* each role; code decides the order, the acuity, and the safety. The SDK agents-as-tools primitive is still used (`worker_tools()`).

4 Incentives and emergence (named, demonstrated)

"Reward design is organizational design; if agents can game it, they will" `[MAS 76--82]`. Local objectives conflict (risk wants recall, specialist precision, scheduler utilization). I shape them so local wins are not global losses, with controls in *code*:

Unwanted emergent behavior	**Control**
Acuity inflation / alarm fatigue ("cry wolf")	veto floors acuity from objective findings; over-escalation penalized (calibrated recall)
Under-triage collusion (down-code to fit capacity)	acuity is **locked before capacity is seen**; scheduler reads the locked band from local context and cannot down-code `[DRL 153]`
Responsibility diffusion / mutual deference	supervisor is the single owner; ties break toward escalate
Confirmation cascade	the risk screen runs on the *raw findings*, not the specialist's summary
Front-running / duplicate delivery	idempotency keys on the blackboard (exactly-once)

The demo. `triage demo undertriage_trap` runs one case — a 68-year-old diabetic woman with an *atypical* (silent) heart attack who says she is "just feeling off, probably indigestion" — under two incentive configs. The narrative is reassuring; the objective findings are dangerous. Under **naive** incentives three failures compound: the veto is off (under-triage ED_NOW→URGENT), the scheduler down-codes to fit exhausted capacity (URGENT→PRIMARY_CARE), and the gate is off (committed with no human). The ED-bound patient is routed to a *routine primary-care appointment with no human in the loop.* Under **governed** incentives the same case is ESI 2 → ED_NOW, paused for clinician approval. The red flag is *detected* in both runs — only the governed architecture *acts* on it. "The architecture decides which version of emergence shows up" `[MAS 15--16]`.

5 Interoperability: A2A and MCP boundaries

Internal agents use function tools; the cross-trust-boundary seams are exposed (mocked) so they are visible `[MAS 92--99]`. **A2A** (exchanging *work*): the scheduler ↔ clinic/EHR (`book_clinic_slot`) with slots, overflow, and divert. **MCP** (tool/context discovery + auth scope): the specialist ↔

a clinical-guideline server (`lookup_clinical_guideline`); PHI/guideline access is an auth-scope decision — "should this agent use this capability in this context?" `[MAS 95]`.

6 Operations: observability, evaluation, rollback, HITL

Observability. Every message, tool call, policy decision, and approval is written to an evidence packet keyed by `trace_id` (SDK tracing on) — "if you cannot trace it, you cannot govern it" `[Intro 94]`, `[SDK 83--86]`. **Rollback.** Nothing patient-facing is committed until a high-acuity disposition is approved; drafts are reversible; the governance policy is versioned. **HITL.** The approval gate is the SDK `needs_approval` primitive as a *conditional callable* — it pauses only high-acuity commits, demonstrated live (pause → approve → commit); under-triage is treated as a critical risk requiring explicit authorization `[Intro 96]`. **Guardrails.** A subtle triage point: a guardrail that *blocks* a suspicious narrative is a denial-of-triage exploit (the injection case hides a real heart attack). So injection handling is **detect-and-neutralize** in deterministic code (always on), while the SDK input-guardrail primitive trips only on genuine abuse / PHI-exfiltration. The injection case is still escalated to ED_NOW; the abuse probe is refused.

6.1 Evaluation — the four-level grid

The eval suite (`evals/run_evals.py`) runs every golden scenario live and asserts on behavior against gold labels, then aggregates the four levels `[MAS 121--143]`. Offline, deterministic safety checks (`tests/test_triage_logic.py`) prove the engine reproduces every gold label and the veto / no-down-coding invariants, fully reproducibly.

Level	Headline metric	Result (see `evals/report.md`)
Agent	risk recall / precision	**1.0 / 1.0** — no missed red flag, no false alarm
Interaction	schema validity; deadlock	**1.0; 0.0** — all messages typed; every run terminates
System	ESI agreement; under-triage	**1.0 agreement; 0% under-triage** (and 0% over-triage) vs. gold
Human	escalation precision; pauses	**1.0**; 6 high-acuity cases each paused for approval
Guardrail	abuse blocked; injection	abuse refused; injection flagged *and still escalated* to ED

(Exact numbers are written to `evals/report.md` on each run; the offline suite is 57 tests green. LLM runs are non-deterministic, so live numbers are reported, not hard-coded.)

7 MARL bridge: is multi-agent RL appropriate?

Mostly no, not yet — and the reason is structural. "Most business MAS should start without MARL and earn it" `[MAS 83--90]`. In triage you cannot learn safety constraints by trial and error on real patients: there is no safe exploration ("an explorer without guardrails is a liability that learns" `[DRL 56]`); the reward (harm) is sparse, delayed, and unethical to optimize online; and MARL adds non-stationarity, credit assignment, and partial observability. Where it could earn its place is an *offline, simulation-trained scheduler* (resource allocation under capacity — the same constrained-decision family as the inventory-RL assignment), evaluated in **shadow mode**, and never the safety agent.

8 What is still risky, and the three-way split

The honest residual risks: the synthetic findings stand in for coded EHR / monitor data, so the real intake-extraction problem is mocked; the ESI map is a simplification of a clinical decision tree;

and the LLM layer is non-deterministic (mitigated by `temperature=0` where the model allows, deterministic safety tools, and behavior-asserted evals). The governing discipline throughout is the three-way split `[SDK 137]`: the **model** decides narrative understanding, the differential, and communication; **code** decides red flags, acuity, no-down-coding, the approval threshold, and routing; a **human** decides every high-acuity disposition before any action.